

计算机系统基础 1-24252 试题答案与详细解析

说明：本解析按照题目顺序整理。个别题目存在表述歧义时，已在对应位置注明常见判分口径。

一、选择题答案速查

题号	答案	题号	答案
1	B	7	B
2	A	8	C
3	C	9	D
4	C	10	A
5	A	11	C
6	D	12	B

二、填空题答案速查

题号	答案
1	① 206；② 11001110。若按 8 位补码有符号解释，① 为 -50。
2	0xD6
3	0x56
4	10100010
5	0xCD
6	%rdi 为数组首地址/基址；%rax 为元素下标/索引
7	10^{12} 字节； $2^{40} = 1099511627776$ 字节
8	DRAM；SRAM
9	① result = 0；② $x \& 0x1$ ；③ 统计 x 的二进制表示中 1 的个数
10	依次为：0x00000000000000100；0x0000080C00000306；0xCCCC；0x0000000400000000；0x00000004；0x00000000000000110

三、选择题详细解析

- 答案 B。 -43 的 8 位补码为 $256 - 43 = 213 = 0xD5$ 。扩展成 16 位有符号数时做符号扩展，最高位为 1，因此高 8 位补 1，得到 0xFFD5。
- 答案 A。 $-6 = -1.5 \times 2^2$ 。符号位 $S=1$ ；阶码 $E=127+2=129=10000001$ ；尾数小数部分为 $1000\dots$ ，所以编码为 $11000000110000000000000000000000$ ，即 $11000000110000000000000000000000$ 。
- 答案 C。块大小 $64B = 2^6$ ，所以块内偏移 CO 为 6 位。缓存容量 32KB，块数 $= 32KB/64B = 512$ ；8 路组相联，所以组数 $= 512/8 = 64 = 2^6$ ，组索引 CI 为 6 位。组号 $= (0x601018 \gg 6) \& 0x3F$ 。地址低 12 位为 0x018，去掉低 6 位块偏移后组索引为 0x00。
- 答案 C。gcc -S sum.c 会把 C 源文件编译到汇编代码，通常输出 sum.s。gcc -c 生成目标文件 .o，gcc -E 只做预处理。
- 答案 A。movb 使用 1 字节。 $70 + 80 = 150$ ，150 的十六进制为 0x96，所以 %bl 中的 8 位编码为 0x96。
- 答案 D。data 是全局变量符号，main 是函数符号，static 局部变量 handle 具有静态存储期，通常会出现现在目标文件的本地符号表中；addvec 是函数内部的自动局部变量，分配在栈帧中，不是链接器意义上的符号。注意 dlsym 中的字符串 "addvec" 只是用于动态查找库函数名，不等价于本源文件定义的符号。

7. 答案 B。SRAM 和 DRAM 断电都会丢失数据；同容量 SRAM 成本高；DRAM 需要刷新。错误项是“DRAM 的速度比 SRAM 快”，实际 SRAM 通常比 DRAM 更快。
8. 答案 C。x86-64 栈向低地址增长，栈底在高地址、栈顶在低地址；函数返回后栈帧被释放；超过寄存器传参数量的参数会通过栈传递。静态局部变量不保存在栈帧中，而是保存在静态存储区，如 .data 或 .bss。
9. 答案 D。典型访问速度大致为：寄存器 > 高速缓存 > 主存/内存 > 固态硬盘 > 磁盘。四个选项中，只有“寄存器 > 内存 > 磁盘”的相对顺序正确。
10. 答案 A。条件分支的实现常见有条件控制转移（条件跳转）和条件数据传送（如 cmov）两类。条件码不能被程序直接当作普通寄存器访问；setX 只能把某种条件测试结果写入字节寄存器。条件数据传送不是跳转；当两边计算量很大或存在不适合提前计算的数据依赖时，通常不适合使用条件数据传送。
11. 答案 C。存储层次向下通常容量更大、单位成本更低、速度更慢。设置高速缓存的目的，是利用局部性让常见访问命中缓存，从而降低平均访问时间，提高主存等效访问速度。
12. 答案 B。testq %rsi,%rsi 判断 p 是否为 NULL，若 p==0 直接返回。cmpq %rdi,(%rsi) 计算 *p - a，jge 表示若 *p >= a 则跳过赋值；只有 p 非空且 *p < a 时执行 movq %rdi,(%rsi)，即 *p = a。因此等价于 if (p && *p < a) *p = a。

四、填空题详细解析

1. 0xCE 的二进制为 1100 1110，即 11001110。若只是普通数制转换， $0xCE = 12 \times 16 + 14 = 206$ ；若题目按 8 位补码有符号数解释，由于最高位为 1，数值为 $206 - 256 = -50$ 。建议考试时优先看老师是否强调“补码/有符号”。
2. $-0x2a = -42$ 。8 位补码编码为 $256 - 42 = 214 = 0xD6$ 。
3. 小端序低地址存放低有效字节。0x12345678 从地址 0x1000 开始存为：0x1000:78, 0x1001:56, 0x1002:34, 0x1003:12。因此 0x1001 处为 0x56。
4. 8 位补码表示 -94：先求 $256 - 94 = 162$ ， $162 = 0xA2 = 10100010$ 。
5. short s = 0xABCD，占 2 字节；赋给 char 时只保留低 8 位，得到 0xCD。题目问编码，不需要考虑 char 被解释成正数还是负数。
6. AT&T 地址形式 D(Rb,Ri,S) 的有效地址为 $Rb + Ri \times S + D$ 。本题 (%rdi,%rax,4) 对应 $\%rdi + \%rax \times 4$ ，正好是 int 数组 a[i] 的地址，所以 %rdi 存数组基址，%rax 存下标 i。
7. 磁盘厂商使用十进制前缀：1TB = 10^{12} 字节。操作系统常用二进制前缀：1TiB = 2^{40} 字节 = 1099511627776 字节。
8. 主存/内存通常用 DRAM 实现，容量大、成本较低但需要刷新；高速缓存通常用 SRAM 实现，速度快但成本高、容量小。
9. 汇编先把 %eax 清零，对应 result = 0。每轮把 x 复制到 rdx，取最低位 x & 1，累加到 result，然后 x 右移一位；只要 x 还非 0 就继续循环。因此函数是 population count，即统计 unsigned long x 中二进制 1 的个数。
10. x86-64 为小端序，多字节数据的最低有效字节在低地址。具体计算见下表。

指令	计算过程	结果
movq %rax, %rdx	直接复制寄存器值	0x0000000000000100

movq (%rax), %rdx	读 0x100 开始的 8 字节: 06 03 00 00 0C 08 00 00	0x0000080C00000306
movw 12(%rax), %dx	地址 0x10C, 读 2 字节: CC CC	0xCCCC
movq (%rax,%rcx,4), %rdx	地址 0x100 + 0x4×4 = 0x110, 读 8 字节: 00 00 00 00 04 00 00 00	0x0000000400000000
movl 16(%rax,%rcx), %edx	地址 0x100 + 16 + 0x4 = 0x114, 读 4 字节: 04 00 00 00	0x00000004
lea 8(%rax,%rcx,2), %rdx	只计算有效地址, 不访存: 0x100 + 8 + 0x4×2	0x0000000000000110

五、第三题：强符号与弱符号

结论：

类别	符号
强符号	m1.c 中：x、y、p1；m2.c 中：main
弱符号	m2.c 中：x
补充说明	m1.c 中 static void p2() 是本地符号，不作为外部链接的强/弱全局符号；printf、p1、p2 在 m2.c 中属于外部引用而不是本文件中的定义。

解析：函数定义和已初始化的全局变量是强符号；未初始化的全局变量是弱符号。static 修饰的函数或变量具有内部链接属性，只在本文件内可见。由于 p2 在 m1.c 中被 static 修饰，m2.c 直接调用 p2 在真实链接时会出现无法解析的外部引用。

六、第四题：结构体对齐与重排

原结构体：

```
struct {
    short a;
    int b[2];
    int *next;
    char c;
} sRec;
```

字段	大小	对齐要求	偏移量	说明
a	2	2	0	从 0 开始, 占 0-1
填充	2	-	2-3	使 b 按 4 字节对齐
b[2]	8	4	4	b[0] 偏移 4, b[1] 偏移 8, 占 4-11
填充	4	-	12-15	使 next 按 8 字节对齐
next	8	8	16	占 16-23
c	1	1	24	占 24
尾部填充	7	-	25-31	结构体总大小按最大对齐 8 的倍数补齐

因此原结构体各字段偏移量为：a = 0, b = 4, next = 16, c = 24；整个结构体大小为 32 字节。

一种最小浪费的重排方式：

```
struct {
    int *next;
    int b[2];
    short a;
    char c;
```

```
} sRec2;
```

字段	偏移量	大小	说明
next	0	8	先放 8 字节对齐字段
b[2]	8	8	按 4 字节对齐
a	16	2	按 2 字节对齐
c	18	1	按 1 字节对齐
尾部填充	19-23	5	总大小补到 8 的倍数

新结构体大小为 24 字节。成员实际大小总和为 $2 + 8 + 8 + 1 = 19$ 字节，而结构体最大对齐为 8，所以理论最小总大小至少是 24 字节；上述重排达到最小值。

七、第五题：流水线吞吐量

流水线时钟周期由最慢一级决定，并且每一级还要加流水线寄存器延迟。满负载时，每个时钟周期完成 1 条指令。

(1) 原始情况：每级组合逻辑 100ps，寄存器延迟 20ps，所以时钟周期 $T = 100 + 20 = 120\text{ps}$ 。吞吐量 $= 1 / 120\text{ps} = 10^{12} / 120 \approx 8.33 \times 10^9$ 条/秒 $= 8.33$ GIPS。

(2) B 级组合逻辑变为 120ps 后，最慢一级为 $120 + 20 = 140\text{ps}$ 。吞吐量 $= 1 / 140\text{ps} = 10^{12} / 140 \approx 7.14$ GIPS。

八、第六题：矩阵转置与缓存未命中

代码访问 $b[j][i] = a[i][j]$ 。C 语言二维数组按行优先存储，块大小为 16 字节，可容纳 4 个 int。N 很大，缓存又不能同时保存多行。以下忽略循环变量、指令访问等额外影响，只统计矩阵数据访问。

(1) 固定一个 i，内层 j 循环执行 N 次。访问 $a[i][j]$ 是按行连续访问，每 4 个 int 发生约 1 次未命中，所以 a 约 $N/4$ 次未命中；访问 $b[j][i]$ 是按列写入，相邻访问跨越一整行，几乎每次都会访问新的缓存块，所以 b 约 N 次未命中。因此一整轮内层循环约 $N/4 + N = 5N/4$ 次未命中；折合每次循环体迭代平均约 $5/4$ 次未命中。

(2) 外层 i 循环共有 N 轮，所以总未命中次数约为 $N \times 5N/4 = 5N^2/4$ 。

(3) 减少未命中的核心思路是分块/分片转置，而不是简单交换 i、j。简单交换循环只能让 b 连续，却会让 a 变成跨行访问；分块可以让一个小块在缓存中被重复使用。示例思路如下：

```
for (ii = 0; ii < N; ii += B)
    for (jj = 0; jj < N; jj += B)
        for (i = ii; i < min(ii + B, N); i++)
            for (j = jj; j < min(jj + B, N); j++)
                b[j][i] = a[i][j];
```

B 应选择为能够让 a、b 的小块尽量留在缓存中的大小。这样 a 的行访问仍有空间局部性，b 的同一小块也能在短时间内被反复命中。

九、第七题：三维数组求和的局部性

```
int sum_array_3d(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (i = 0; i < N; i++)
```

```

        for (j = 0; j < N; j++)
            for (k = 0; k < M; k++)
                sum += a[k][i][j];
    return sum;
}

```

(1) 数据空间局部性较差。C 语言多维数组按行优先存储，最后一维 j 连续。而原程序的最内层循环改变的是 k ，访问 $a[k][i][j]$ 时相邻两次访问跨越 $N \times N$ 个 `int`，步长很大，不能充分利用同一缓存块中的相邻元素。

(2) 指令时间局部性很好。三重循环的循环控制指令和 `sum += ...` 语句会被反复执行，指令本身集中且重复，因此指令缓存容易命中。

(3) 有更好的局部性实现方式：把最后一维 j 放到最内层，使访问顺序与内存连续布局一致。

```

int sum_array_3d_good(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (k = 0; k < M; k++)
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                sum += a[k][i][j];
    return sum;
}

```

这样最内层访问 $a[k][i][0]$, $a[k][i][1]$, ..., 地址连续，每个缓存块读入后可连续使用多个 `int`，空间局部性明显更好。

十、第八题：x86-64 过程调用规范

以下按常见的 x86-64 Linux System V ABI 说明。

1. 控制流转移：调用者执行 `call` 指令时，CPU 会把返回地址压入栈中，然后跳转到被调用函数入口。被调用函数执行结束时通过 `ret` 指令从栈顶弹出返回地址，并跳回调用点之后继续执行。
2. 数据传输：整型和指针参数通常按顺序放入 `%rdi`、`%rsi`、`%rdx`、`%rcx`、`%r8`、`%r9`；超过 6 个的参数通过栈传递。返回值通常放在 `%rax` 中；较大的结构体返回可能通过隐藏指针或内存完成。浮点参数一般通过 XMM 寄存器传递。
3. 寄存器保存约定：调用者保存寄存器包括 `%rax`、`%rcx`、`%rdx`、`%rsi`、`%rdi`、`%r8-%r11`，函数调用前如果调用者还要用这些值，需要自己保存。被调用者保存寄存器包括 `%rbx`、`%rbp`、`%r12-%r15`，被调用函数若修改它们，返回前必须恢复。`%rsp` 始终用于栈顶管理。
4. 过程中的数据存储管理：栈向低地址增长。函数的栈帧通常保存返回地址、旧的帧指针、被调用者需要保存的寄存器、局部变量、临时溢出值以及传给其他函数的栈上传参。编译器可以使用 `%rbp` 作为帧指针，也可以省略帧指针，直接基于 `%rsp` 访问栈帧。静态变量和全局变量不在栈帧中，而在静态存储区；动态分配对象在堆中。
5. 栈对齐：函数调用边界通常要求栈按 16 字节对齐，以满足 ABI 和 SIMD 指令访问要求。编译器会在函数序言和调用前调整 `%rsp`。

十一、第九题：Linux 文件复制命令

当前工作目录为 /home/user，main.c 已在当前目录下，目标目录 mylab 不存在。命令如下：

```
mkdir mylab
cp main.c mylab/
```

也可以写成绝对路径，更稳妥：

```
mkdir -p /home/user/mylab
cp /home/user/main.c /home/user/mylab/
```

十二、第十题：组相联高速缓存地址划分与访问命中

已知地址位宽 12 位，块大小 4 字节，所以块内偏移 CO 需要 $\log_2(4)=2$ 位；共有 4 个组，所以组索引 CI 需要 $\log_2(4)=2$ 位；剩余 8 位为标记 CT。

位号	11	10	9	8	7	6	5	4	3	2	1	0
字段	CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CO	CO

因此 CT = 地址[11:4]，CI = 地址[3:2]，CO = 地址[1:0]。

访问 0x831：

0x831 的 CT = 0x83，CI = 0，CO = 1。查看第 0 组，有一个有效块标记为 83，字节序列为 FE 97 CC D0，因此命中，偏移 1 处读出的值为 0x97。

访问 0x833：

0x833 的 CT = 0x83，CI = 0，CO = 3。仍然对应第 0 组标记 83 的有效块，因此写命中。题目没有给出要写入的数据，所以写入后的值无法确定；如果只问写入前该地址缓存中的原字节，则偏移 3 的原值为 0xD0。

操作	地址	命中	读出的值/说明
读	0x831	Y	0x97
写	0x833	Y	写入值题目未给，故读出值不适用/未知；写前原字节为 0xD0

复习提示

本套题高频考点集中在：补码与符号扩展、IEEE 754、cache 地址划分、x86-64 汇编寻址与条件跳转、链接符号强弱、结构体对齐、流水线吞吐量、局部性优化以及 System V ABI 过程调用规范。建议把第 3、10 题这类 cache 位划分题，以及第 12 题 cmp/jcc 方向题重点练熟。